Lab Procedure
Microphone & Speaker

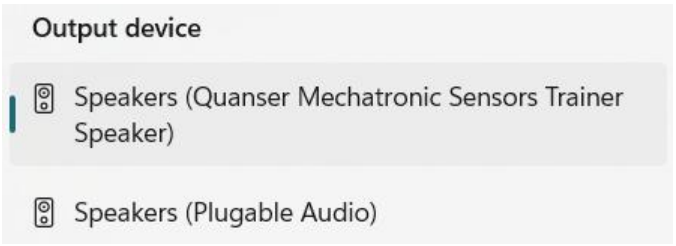
Introduction

Ensure the following:

1. You have reviewed the Application Guide – Microphone & Speaker
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.

Write Sine Wave to Speaker

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **`File > Open Folder`**..., and browse to the directory containing the python files and lab procedure for the Microphone & Speaker lab (…/sensors_trainer/ 1_fundamentals/ microphone_speaker/hardware/python). Open this directory.
2. Open speaker.py. In this script, you will complete **generate_waveform()** function to generate a sine wave of a specified frequency. Complete this function to create a sound buffer with desired frequency, sample rate and duration. Make sure the return **wave** variable is in the proper format and shape.
3. When the function is complete, run the script using the ▷ button or the terminal. Enter the "1000" in the terminal when prompted. Does the speaker produce an expected sound? If not ensure the following before debugging your function:
    a. You have selected the Sensor Trainer as the output device as shown below.



    b. Your speaker volume is at an audible level.

4. Start from 200 Hz and gradually increase the frequency to 1500 Hz. What do you notice about the amplitude as the frequency increases? Check the datasheet of the speaker and note down anything that could explain your observations.

5. Stop the script using **`Ctrl+C`** when you are finished.

Visualizing Audio Data

6. In the same directory, open microphone_visualization.py. Run the script using the  button or the terminal. A window labeled "Audio Visualizer" will appear, showing real-time microphone data.
7. Gently tap the housing of the sensor trainer, does the visualizer respond as expected? Click "X" to close the pop-up window and stop the script.
8. During the instantiation of the visualizer, parameters including **sample_rate**, **block_size**, **waveform_max_time_blocks**, and **waveform_yrange** are passed to configure the display. Change **sample_rate** to **48000**, what changes do you observe in the visualizer? What do these changes imply about the quality and resolution of the audio data?
9. Now, change **block_size** to 2048, and then 4096. How does this parameter affect the visualizer's responsiveness?
10. **waveform_max_time_blocks** and **waveform_yrange** affect only the visual appearance of the waveform. Modify these values and note how they change the visual display.
11. The analysis of sound data is often conducted in the frequency domain, as such, fast Fourier transform (FFT) is often performed on sound data to better display the properties of the sounds. Change **sample_rate** back to 44100 and **block_size** 1024. Uncomment the additional arguments in the visualizer and run the script again. Is FFT curve behaving as expected?
12. Change **fft_block_size** to 8*1024, how does it affect the FFT display? Comment its behaviour in terms of smoothness, frequency resolution and update rate.
13. Similarly, **fft_xrange** and **fft_yrange** only affects visualization. Adjust them to focus on regions of interest and note their effects.
14. Open a new terminal, such that you can run microphone_visualization.py and speaker.py simultaneously. You can increase the **wave_duration** variable in speaker.py for easier observation. Use speaker.py to play a sine wave at 1000 Hz. Does the wave form show an expected shape? Does the FFT curve show a peak at the expected location? Record your observations
15. Use speaker.py to play different frequencies between 500 Hz and 1500 Hz, record your observations from both waveform and FFT plots. Adjust **waveform_max_time_blocks**, **waveform_yrange**, **fft_xrange** and **fft_yrange** to highlight key features.
16. When finished, close the application by clicking "X" on the visualizer window.

Microphone Application – Clap Detection

17. In the same directory, open microphone_clap_detect.py and locate the **clap_detection** class. This class will be passed to the visualizer as a callback function and will be called automatically whenever a new block of microphone data is available. Using a callable class allows you to store persistent information as attributes.

18. Complete the clap detection callback by analyzing the incoming audio block. You may compute the maximum absolute amplitude (or RMS value of the block) and compare it to a threshold value.

19. Run the script using the ▷ button or the terminal, and clap near the microphone. Do you see "Clap detected" printed in the terminal when you clap? Adjust the detection threshold to improve performance.

20. Implement a debounce period to prevent multiple detections for one clap event. You can add attributes in the **__init__()** function of **clap_detection** to register time stamps of last detection.

21. Observe how the output changes with different types of sounds (voice, tapping, etc.)

22. Stop the script by clicking the "X" on the visualizer window.

Microphone Application – Frequency Detection

23. Open microphone_frequency_detect.py in the same directory. This script is similar to clap detection but includes an incomplete callback class for frequency detection using FFT data.

24. Navigate to the **frequency_detection** callback class, this callback will be used to process the FFT results. Complete this callback to identify dominant peaks in the FFT curve and utilize **previous_peaks** attribute to only print detected peaks when they change. Consider using off-the-shelf libraries such as **find_peaks** from **scipy.signal** for this task.

25. Run the script using the ▷ button or the terminal. Then in a separate terminal, run speaker.py to play a sine wave in the range of 500-1500 Hz. Does the frequency detection correctly report the played sine wave? Record your observations.

26. Modify the **fft_block_size** and observe how it affects detection stability and responsiveness.

27. Stop your program by clicking "X" on the visualizer window, save and close your program.

28. Power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.